

BÁO CÁO BÀI TẬP: KỸ NĂNG "PROMPT ENGINEERING" VÀ ỨNG DỤNG TRONG HỌC TẬP

Họ và tên sinh viên: Trần Đức Mạnh

Mã số sinh viên: 25020258

Công cụ AI sử dụng thử nghiệm: ChatGPT (hoặc Google Gemini/Copilot)

I. MỞ ĐẦU

Trong kỷ nguyên của các Mô hình Ngôn ngữ Lớn (LLMs), chất lượng câu trả lời của AI phụ thuộc trực tiếp vào chất lượng của câu lệnh (Prompt). Báo cáo này trình bày quá trình áp dụng các kỹ thuật "Prompt Engineering" từ cơ bản đến nâng cao vào 3 tác vụ học tập cốt lõi của sinh viên khối ngành kỹ thuật/công nghệ. Mục tiêu là phân tích sự khác biệt trong đầu ra, từ đó đúc kết các nguyên tắc giao tiếp hiệu quả với AI.

II. LỰA CHỌN VÀ PHÂN TÍCH TÁC VỤ HỌC TẬP

Để minh họa rõ nét, tôi lựa chọn 3 tác vụ xoay quanh các môn học mang tính tư duy logic cao:

1. Tác vụ 1: Tóm tắt một tài liệu học thuật (Môn Cấu trúc Dữ liệu & Giải thuật)

- Mục tiêu:* Nắm bắt nhanh ý tưởng cốt lõi của một thuật toán phức tạp mà không bị lạc vào các chi tiết mã nguồn.
- Thách thức:* AI dễ đưa ra tóm tắt quá chung chung (thiếu ý nghĩa thực tiễn) hoặc sao chép y nguyên ngôn ngữ hàn lâm khó hiểu.

2. Tác vụ 2: Giải thích một khái niệm phức tạp (Môn Đại số tuyến tính)

- Mục tiêu:* Hiểu được bản chất hình học và ý nghĩa thực tế của một khái niệm toán học trừu tượng.
- Thách thức:* Các định nghĩa toán học thường khô khan. Nếu AI chỉ dùng công thức để giải thích, sinh viên sẽ không nắm được "trực giác" đằng sau khái niệm đó.

3. Tác vụ 3: Tạo bộ câu hỏi ôn tập (Môn Lập trình Hướng đối tượng - OOP)

- Mục tiêu:* Hệ thống hóa kiến thức và rèn luyện kỹ năng giải quyết vấn đề qua các tình huống thực tế.
- Thách thức:* AI thường tạo ra các câu hỏi trắc nghiệm học thuộc lòng (rất nông) thay vì các bài tập yêu cầu tư duy thiết kế hệ thống.

III. XÂY DỰNG, THỬ NGHIỆM VÀ SO SÁNH CÁC PHIÊN BẢN PROMPT

1. Tác vụ 1: Tóm tắt thuật toán Dijkstra (Cấu trúc dữ liệu & Giải thuật)

- Prompt Cơ bản:** "Tóm tắt thuật toán Dijkstra."

- **Prompt Cải tiến:** "Hãy tóm tắt thuật toán tìm đường đi ngắn nhất Dijkstra. Nêu rõ đầu vào, đầu ra, và các bước thực hiện cơ bản."
- **Prompt Nâng cao (Áp dụng Role-prompting, Formatting, Constraint):** "Đóng vai một Giảng viên Khoa học Máy tính nhiệt huyết. Hãy tóm tắt thuật toán Dijkstra cho sinh viên năm 2 theo cấu trúc sau: 1. Ý tưởng cốt lõi (sử dụng một ẩn dụ đời thực để minh họa). 2. Các bước thực hiện (bullet points ngắn gọn). 3. Độ phức tạp thời gian & không gian. 4. Hai ứng dụng thực tế phổ biến."
- [KẾT QUẢ PROMPT CƠ BẢN]

"Tóm tắt thuật toán Dijkstra."

Thuật toán **Dijkstra** (do Edsger W. Dijkstra đề xuất) dùng để tìm **đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh còn lại** trong đồ thị có trọng số **không âm**.

Ý tưởng chính

Luôn chọn đỉnh **chưa xét có khoảng cách tạm thời nhỏ nhất**, rồi "cố định" khoảng cách đó là tối ưu.

Các bước cơ bản

- Khởi tạo**
 - Gán khoảng cách từ nguồn đến chính nó = 0
 - Các đỉnh khác = ∞
 - Tập đỉnh chưa xét chứa tất cả các đỉnh
- Lặp**
 - Chọn đỉnh u chưa xét có khoảng cách nhỏ nhất
 - Đánh dấu u đã xét
 - Với mỗi đỉnh kề v của u :
 - Nếu đi qua u mà ngắn hơn:
- Kết thúc**
 - Khi tất cả đỉnh đã được xét hoặc không còn cập nhật

$$d[v] = \min(d[v], d[u] + w(u, v))$$

- [KẾT QUẢ PROMPT CẢI TIẾN]

"Hãy tóm tắt thuật toán tìm đường đi ngắn nhất Dijkstra. Nêu rõ đầu vào, đầu ra, và các bước thực hiện cơ bản."

Dưới đây là bản tóm tắt **chuẩn – rõ ràng – đúng format** của thuật toán Dijkstra:

1. Giới thiệu

Thuật toán Dijkstra (của Edsger W. Dijkstra) dùng để tìm **đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh khác** trong đồ thị có **trọng số không âm**.

2. Đầu vào (Input)

- Đồ thị $G = (V, E)$
 - V : tập đỉnh
 - E : tập cạnh có trọng số $w(u, v) \geq 0$
- Đỉnh nguồn $s \in V$

3. Đầu ra (Output)

- Mảng khoảng cách $d[v]$: độ dài đường đi ngắn nhất từ s đến mọi đỉnh v
- (Tuỳ chọn) Mảng truy vết ***parent[v]***: dùng để khôi phục đường đi

4. Các bước thực hiện

Bước 1: Khởi tạo

- Với mọi đỉnh v :
$$d[v] = \infty$$
- Đặt:
$$d[s] = 0$$
- Tạo tập đỉnh chưa xét (hoặc hàng đợi ưu tiên)

Bước 2: Lặp

Trong khi còn đỉnh chưa xét:

- Chọn đỉnh u có $d[u]$ nhỏ nhất
- Đánh dấu u đã xét
- Với mỗi đỉnh kề v của u :
 - Nếu:
$$d[u] + w(u, v) < d[v]$$

thì cập nhật:
$$d[v] = d[u] + w(u, v)$$

và:

Hỏi bất kỳ điều gì

ChatGPT có thể mắc lỗi. Hãy kiểm tra các thông tin quan trọng. Vui lòng tham khảo Tùy chọn cookie.

• [KẾT QUẢ PROMPT NÂNG CAO]

"Đóng vai một Giảng viên Khoa học Máy tính nhiệt huyết. Hãy tóm tắt thuật toán Dijkstra cho sinh viên năm 2 theo cấu trúc sau: 1. Ý tưởng cốt lõi (sử dụng một ẩn dụ đời thực để minh họa). 2. Các bước thực hiện (bullet points ngắn gọn). 3. Độ phức tạp thời gian & không gian. 4. Hai ứng dụng thực tế phổ biến." Được rồi, mình sẽ giải thích như đang đứng lớp nhé — ngắn gọn nhưng "thấm" 🙌 1. Ý tưởng cốt lõi (ẩn dụ đời thực) Hãy tưởng tượng bạn đang ở một thành phố và muốn đi từ nhà đến tất cả các địa điểm khác sao cho tốn ít thời gian nhất. Bạn làm gì? 👉 Mỗi lần, bạn luôn chọn địa điểm gần nhất mà bạn biết chắc đường đi ngắn nhất rồi, rồi từ đó thử đi tiếp đến các nơi khác. 💡 Thuật toán Dijkstra (của Edsger W. Dijkstra) cũng làm đúng như vậy: • Luôn "chốt" đỉnh gần nhất trước • Sau đó dùng nó để cải thiện đường đi đến các đỉnh khác → Giống như lan tỏa sóng từ nguồn ra ngo từng lớp một.	2. Các bước thực hiện (ngắn gọn) • Khởi tạo: • $d[s] = 0$, các đỉnh khác $= \infty$ • Lặp cho đến khi hết đỉnh: • Chọn đỉnh u có khoảng cách nhỏ nhất (chưa xét) • Đánh dấu u đã xét • Với mỗi đỉnh kề v: • Nếu đi qua u ngắn hơn: • Cập nhật $d[v] = d[u] + w(u, v)$ • Lưu cha: $parent[v] = u$ 3. Độ phức tạp • Thời gian: • $O(V^2)$: nếu dùng mảng thường • $O((V + E) \log V)$: nếu dùng priority queue (heap) • Không gian: • $O(V)$ cho mảng khoảng cách + parent • $O(V + E)$ nếu tính cả lưu đồ thị 4. Hai ứng dụng thực tế phổ biến 1. Bản đồ & định vị
a	b

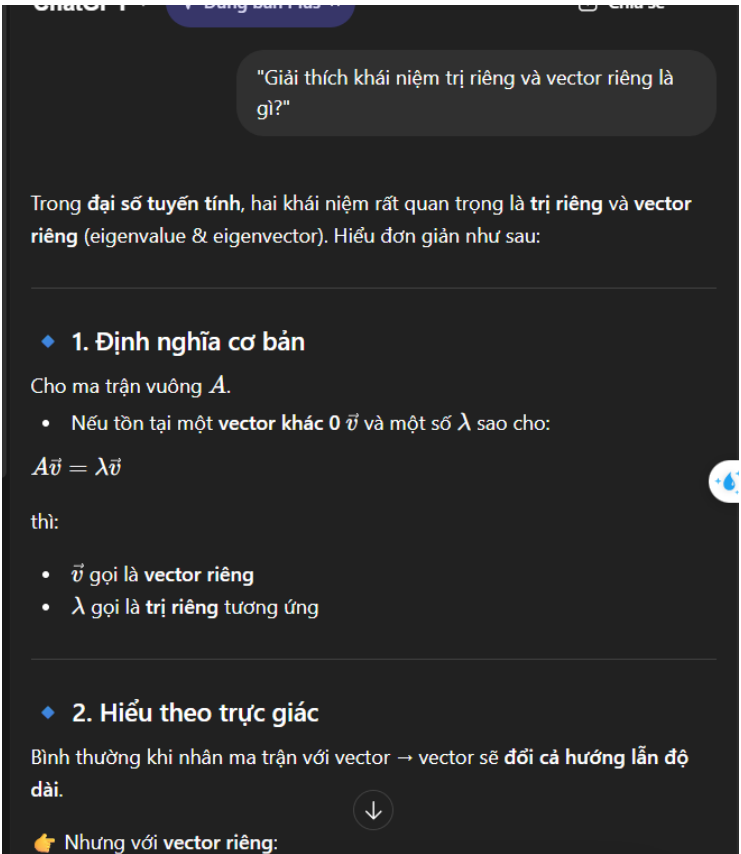
4. Hai ứng dụng thực tế phổ biến 1. Bản đồ & định vị • Google Maps, GPS • Tìm đường đi nhanh nhất giữa các địa điểm 2. Mạng máy tính • Định tuyến gói tin (routing) • Các giao thức như OSPF sử dụng biến thể của Dijkstra Chốt lại như giảng viên hay nói: "Dijkstra = luôn tham lam chọn đường ngắn nhất hiện tại, nhưng kết quả lại là tối ưu toàn cục — miễn là không có cạnh âm."

Nhận xét sắc thái kết quả: Phiên bản cơ bản chỉ trả về định nghĩa từ điển. Phiên bản cải tiến đã có cấu trúc nhưng vẫn khô khan. Phiên bản nâng cao mang lại hiệu quả vượt trội: sự xuất hiện của "ẩn dụ đời thực" (như việc tìm đường đi trên bản đồ Google Maps) giúp thuật toán trở nên trực quan, cấu trúc rõ ràng giúp dễ dàng ghi chép vào sổ tay.

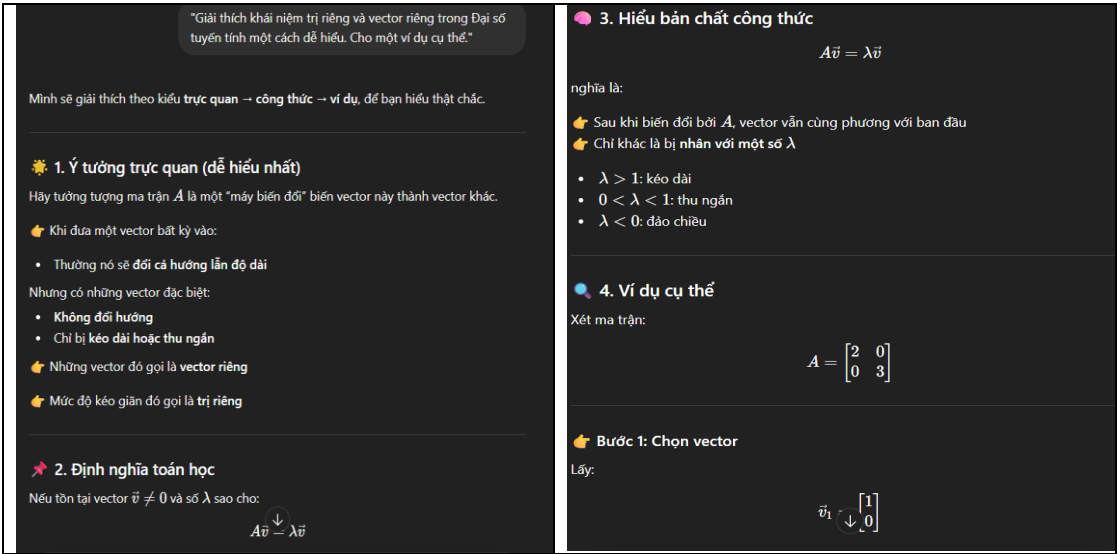
2. Tác vụ 2: Giải thích Trị riêng (Eigenvalues) & Vector riêng (Eigenvectors)

- **Prompt Cơ bản:** "Giải thích khái niệm trị riêng và vector riêng là gì?"
- **Prompt Cải tiến:** "Giải thích khái niệm trị riêng và vector riêng trong Đại số tuyến tính một cách dễ hiểu. Cho một ví dụ cụ thể."

- **Prompt Nâng cao (Áp dụng Chain-of-Thought, Analogy, Audience Targeting):**
"Đóng vai một gia sư Toán cao cấp. Giải thích khái niệm Trị riêng (Eigenvalues) và Vector riêng (Eigenvectors) cho một sinh viên đang bị mất gốc môn Đại số tuyến tính. *Hãy suy nghĩ theo từng bước (Chain of Thought):* Bắt đầu bằng trực giác hình học (sự kéo giãn không gian) -> Định nghĩa toán học đơn giản nhất -> Ứng dụng thực tế (ví dụ: Thuật toán PageRank của Google). Tránh lạm dụng công thức mà trộn phức tạp ở phần mở đầu."
- [KẾT QUẢ PROMPT CƠ BẢN]



- [KẾT QUẢ PROMPT CẢI TIẾN]



- [KẾT QUẢ PROMPT NÂNG CAO]

"Đóng vai một gia sư Toán cao cấp. Giải thích khái niệm Trị riêng (Eigenvalues) và Vector riêng (Eigenvectors) cho một sinh viên đang bị mất gốc môn Đại số tuyến tính. Hãy suy nghĩ theo từng bước (Chain of Thought): Bắt đầu bằng trực giác hình học (sự kéo giãn không gian) -> Định nghĩa toán học đơn giản nhất -> Ứng dụng thực tế (ví dụ: Thuật toán PageRank của Google). Tránh lạm dụng công thức ma trận phức tạp ở phần mở đầu."

Được, mình sẽ giải thích như đang kèm 1-1 cho bạn, đi từ **trực giác** → **định nghĩa** → **ứng dụng**, không làm bạn "ngợp" công thức.

🔴 **BƯỚC 1: Trực giác hình học (hiểu cái bản chất trước)**

Hãy tưởng tượng:

- 👉 Bạn có một tấm cao su (mặt phẳng)
- 👉 Ma trận A là một "lực" làm biến dạng tấm đó

Khi bạn vẽ nhiều mũi tên (vector) trên tấm:

- Đa số mũi tên sẽ:
 - bị **xoay hướng**
 - bị **kéo dài / thu ngắn**

Nhưng... có một số mũi tên rất "đặc biệt":

👉 Sau khi biến đổi:

- **KHÔNG** bị xoay hướng
- Chỉ bị **kéo dài hoặc co lại**

💡 **Ý nghĩa cực kỳ quan trọng:**

- Những hướng "không bị xoay" → **vector riêng**
- Mức độ kéo giãn → **trị riêng**

🔴 **BƯỚC 2: Định nghĩa toán học (giản lược nhất có thể)**

Ta chỉ cần nhớ 1 dòng:

$$A\vec{v} = \lambda\vec{v}$$

Hiểu đơn giản:

- 👉 Áp dụng biến đổi A lên vector $\vec{v}$
- 👉 Kết quả chỉ là **nhân vector đó với một số λ**

✓ $\vec{v}$ → vector riêng (hướng không đổi)
 ✓ λ → trị riêng (độ kéo giãn)

🔍 **BƯỚC 3: Ví dụ cực đơn giản**

Xét phép biến đổi:

- Trục ngang (x) bị nhân 2
- Trục dọc (y) bị nhân 3

👉 **Vector nằm ngang:**

$$\vec{v} = (1, 0)$$

Sau biến đổi → thành (2,0)

- ✓ Không đổi hướng
- 👉 Đây là **vector riêng**
- 👉 Trị riêng = 2

👉 **Vector thẳng đứng:**

$$\vec{v} = (0, 1)$$

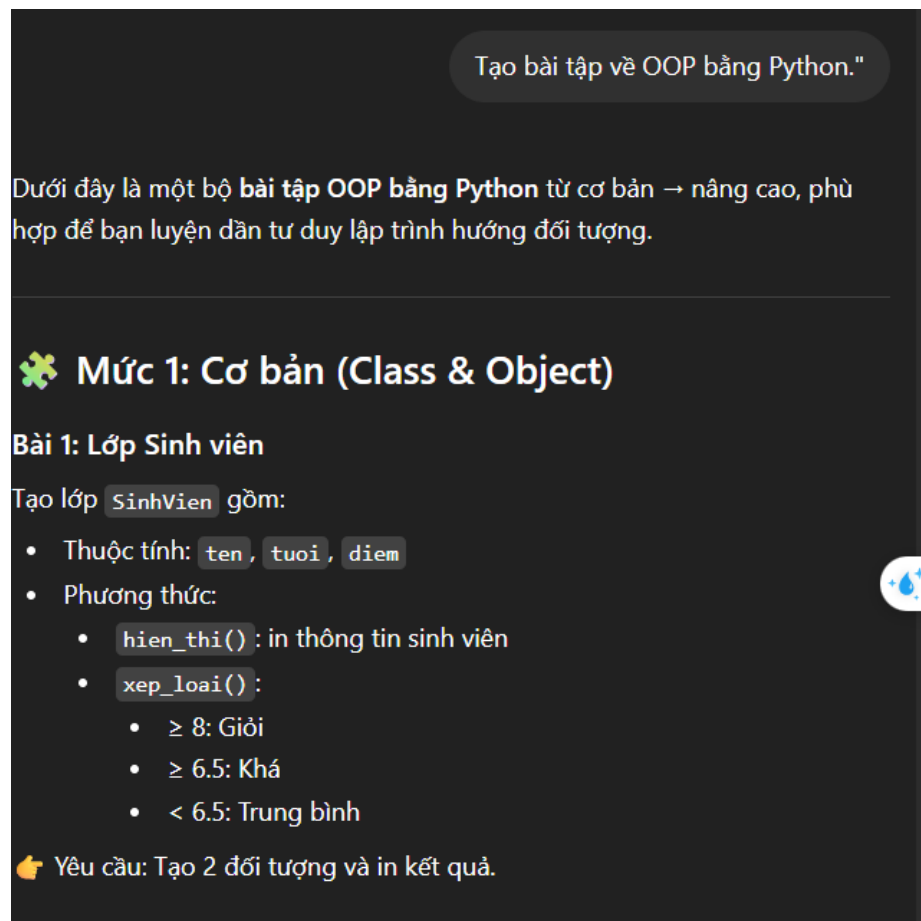
→ thành (0,3)

- ✓ Không đổi hướng
- 👉 Trị riêng = 3

Nhận xét sắc thái kết quả: AI ở prompt nâng cao đã thực sự "nhập vai" gia sư. Thay vì ném ra công thức định thức ngay lập tức như prompt cơ bản, AI bắt đầu bằng hình ảnh tưởng tượng kéo giãn một tấm bìa cao su. Kỹ thuật Chain-of-Thought dẫn dắt luồng suy nghĩ của AI đi từ trực giác đến hàn lâm, giải quyết triệt để vấn đề "khó hiểu" của tác vụ.

3. Tác vụ 3: Tạo bộ câu hỏi ôn tập Lập trình Hướng đối tượng (OOP) bằng Python

- **Prompt Cơ bản:** "Tạo bài tập về OOP bằng Python."
- **Prompt Cải tiến:** "Tạo 5 câu hỏi lý thuyết và 1 bài tập thực hành viết code về Lập trình hướng đối tượng trong Python (tập trung vào Kế thừa và Đa hình)."
- **Prompt Nâng cao (Áp dụng Role-prompting, Output Formatting, Few-shot style):** "Đóng vai một chuyên gia phỏng vấn kỹ thuật (Technical Interviewer). Hãy tạo một bộ tài liệu ôn tập OOP bằng Python. Tuân thủ cấu trúc sau:
 - Phần 1: 3 câu hỏi phỏng vấn tình huống (Behavioral/Scenario-based) kiểm tra hiểu biết về Kế thừa, Đóng gói, Đa hình. Trình bày dưới dạng [Bảng: Câu hỏi | Mức độ khó | Gợi ý trả lời].
 - Phần 2: 1 bài tập thiết kế hệ thống nhỏ (Ví dụ: Thiết kế class cho hệ thống quản lý thư viện). Cung cấp yêu cầu đề bài và đoạn code Python mẫu chứa đáp án."
- [KẾT QUẢ PROMPT CƠ BẢN]



- [KẾT QUẢ PROMPT CẢI TIẾN]

Tạo 5 câu hỏi lý thuyết và 1 bài tập thực hành viết code về Lập trình hướng đối tượng trong Python (tập trung vào Kế thừa và Đa hình)."

Dưới đây là bộ bài tập **tập trung vào Kế thừa (Inheritance) và Đa hình (Polymorphism)** trong OOP Python:

I. Câu hỏi lý thuyết (5 câu)

Câu 1

Kế thừa (Inheritance) là gì?
→ Giải thích lợi ích của kế thừa trong lập trình hướng đối tượng.

Câu 2

Phân biệt:

- Kế thừa đơn
- Kế thừa đa cấp
- Kế thừa đa kế thừa

Cho ví dụ minh họa ngắn gọn cho mỗi cái.

Câu 3

Đa hình (Polymorphism) là gì?
→ Giải thích tại sao các lớp con có thể ghi đè (override) phương thức của lớp cha.

Câu 4

So sánh:

- Method Overriding
- Method Overloading (Python hỗ trợ như thế nào?)

Câu 5

Cho đoạn code sau, hãy giải thích kết quả:

```
Python
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

a = Animal()
b = Dog()

a.speak()
b.speak()
```

Câu hỏi:

- Output là gì?
- Tại sao lại như vậy?

[KẾT QUẢ PROMPT NÂNG CAO]

Phần 1: 3 câu hỏi phỏng vấn tình huống (Behavioral/Scenario-based) kiểm tra hiểu biết về Kế thừa, Đóng gói, Đa hình. Trình bày dưới dạng [Bảng: Câu hỏi | Mức độ khó | Gợi ý trả lời].

Phần 2: 1 bài tập thiết kế hệ thống nhỏ (Ví dụ: Thiết kế class cho hệ thống quản lý thư viện). Cung cấp yêu cầu đề bài và đoạn code Python mẫu chứa đáp án."

Dưới đây là bộ tài liệu ôn tập theo phong cách **phỏng vấn kỹ thuật**, tập trung vào OOP trong Python:

PHẦN 1: Câu hỏi phỏng vấn tình huống

Câu hỏi	Mức độ khó	Gợi ý trả lời
Bạn đang xây dựng hệ thống quản lý nhân viên. Có nhiều loại nhân viên (Full-time, Part-time) với cách tính lương khác nhau. Bạn sẽ dùng kế thừa như thế nào để thiết kế?	Trung bình	Tạo lớp cha <code>NhanVien</code> chứa thuộc tính chung (tên, lương cơ bản). Các lớp con kế thừa và override phương thức <code>inh_luong()</code> . Điều này giúp tái sử dụng code và dễ mở rộng.
Một lớp <code>TaiKhoanNganHang</code> có thuộc tính số dư. Làm thế nào để đảm bảo người dùng không thể truy cập trực tiếp và sửa số dư một cách tùy ý?	Cơ bản	Sử dụng đóng gói (encapsulation) : đặt thuộc tính <code>_so_du</code> là private, cung cấp phương thức <code>nap_tien()</code> , <code>rut_tien()</code> . Có thể dùng getter để đọc nhưng không cho sửa trực tiếp.
Bạn có một danh sách các đối tượng: <code>Dog</code> , <code>Cat</code> , <code>Bird</code> . Mỗi con vật có cách kêu khác nhau. Làm sao để duyệt danh sách và gọi đúng hành vi mà không cần kiểm tra <code>if-else</code> ?	Trung bình	Áp dụng đa hình (polymorphism) : tất cả các lớp đều có phương thức <code>speak()</code> . Khi duyệt list, chỉ cần gọi <code>animal.speak()</code> → Python sẽ tự gọi đúng phương thức của từng lớp (dynamic dispatch).

PHẦN 2: Bài tập thiết kế hệ thống

Đề bài: Thiết kế hệ thống quản lý thư viện

Yêu cầu:

Xây dựng hệ thống gồm các thành phần:

- Lớp Book**
 - Thuộc tính: `title`, `author`, `is_available`
 - Phương thức:
 - `borrow()`
 - `return_book()`
- Lớp User**
 - Thuộc tính: `name`, `borrowed_books`
 - Phương thức:
 - `borrow_book(book)`
 - `return_book(book)`
- Lớp Library**
 - Thuộc tính: danh sách sách
 - Phương thức:
 - `add_book(book)`
 - `show_books()`

Yêu cầu OOP:

- Dùng Encapsulation để bảo vệ trạng thái sách
- Có thể mở rộng bằng Inheritance (ví dụ: `Student`, `Teacher`)
- Áp dụng Polymorphism nếu mở rộng hành vi mượn sách

Code mẫu (đáp án tham khảo)

```
Python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.__is_available = True # encapsulation

    def borrow(self):
        if self.__is_available:
            self.__is_available = False
            return True
        return False

    def return_book(self):
        self.__is_available = True

    def is_available(self):
        return self.__is_available
```

Gợi ý mở rộng (đúng kiểu interview thích hợp)

- Thêm lớp `Student(User)` giới hạn số sách mượn
- Thêm `Teacher(User)` mượn nhiều hơn -- đa hình
- Thêm xử lý exception
- Dùng `@property` thay cho getter truyền thống

Hỏi bất kỳ điều gì

Nhận xét sắc thái kết quả: Prompt nâng cao ép buộc AI phải xuất dữ liệu dưới dạng Bảng, giúp người học cực kỳ dễ nhìn. Hơn nữa, thay vì ra câu hỏi học vẹt "Đa hình là gì?", việc đóng vai người phỏng vấn khiến AI tạo ra các câu hỏi tình huống thực chiến, nâng cao chất lượng ôn tập lên mức độ vận dụng cao.

IV. PHÂN TÍCH LÝ DO HIỆU QUẢ CỦA PROMPT NÂNG CAO

Qua thử nghiệm, có thể thấy Prompt Nâng cao vượt trội hoàn toàn vì chúng tác động vào cơ chế hoạt động cốt lõi của mô hình Transformer:

- Cung cấp "Ngữ cảnh" (Context & Role):** AI không có tư duy chủ động. Việc đóng vai (Role-prompting) giúp thu hẹp không gian xác suất của các từ vựng. Khi đóng vai "Giáo sư toán", AI tự động loại bỏ các thuật ngữ học thuật khô khan và tăng trọng số (weights) cho các từ ngữ mang tính diễn giải, khuyến khích.
- Kỹ thuật "Chain-of-Thought" (Suy nghĩ từng bước):** Yêu cầu AI "bắt đầu bằng trực giác -> định nghĩa -> ứng dụng" ép mô hình phải tạo ra một chuỗi suy luận logic. Điều này giảm thiểu hiện tượng "ảo giác" (hallucination) và đảm bảo tính mạch lạc của văn bản học thuật.
- Thiết lập "Ràng buộc" (Constraints & Formatting):** Các chỉ thị rõ ràng về định dạng (dùng bullet points, tạo bảng, giới hạn số câu) đóng vai trò như các hàng rào, buộc AI phải định hình luồng thông tin đầu ra thay vì sinh chữ một cách tự do, lan man.

V. TỔNG HỢP NGUYÊN TẮC VÀ MẸO VIẾT PROMPT HIỆU QUẢ

Dựa trên các phân tích trên, để tận dụng tối đa AI trong học tập, tôi đề xuất mô hình **C.R.E.A.T.E** khi viết prompt:

- **C - Context (Ngữ cảnh):** Cung cấp nền tảng. (VD: "Tôi là sinh viên năm 2 đang mất gốc môn Đại số tuyến tính...").
- **R - Role (Đóng vai):** Xác định chuyên môn cho AI. (VD: "Hãy hành xử như một kỹ sư phần mềm cấp cao...").
- **E - Exact Task (Nhiệm vụ cụ thể):** Nêu rõ việc cần làm. (VD: "Tóm tắt chương 1", "Sửa lỗi đoạn code này").
- **A - Actionable Steps (Các bước thực hiện):** Hướng dẫn tư duy từng bước (Chain-of-thought) để AI có tư duy logic.
- **T - Tone & Format (Giọng điệu và Định dạng):** Yêu cầu cách trình bày. (VD: "Dùng giọng điệu học thuật", "Trình bày thành bảng 3 cột").
- **E - Examples (Ví dụ tham khảo / Few-shot):** Đưa ra một mẫu bạn thích để AI bắt chước theo.

VI. KẾT LUẬN

Viết prompt không chỉ là "ra lệnh" cho máy móc, mà là kỹ năng giao tiếp và tư duy hệ thống. Bằng cách áp dụng các kỹ thuật cấu trúc lệnh rõ ràng, thiết lập ngữ cảnh và định dạng, sinh viên có thể biến các công cụ AI từ một cỗ máy tìm kiếm thông thường thành một gia sư cá nhân xuất sắc, hỗ trợ đắc lực cho việc nghiên cứu và tự học chuyên sâu.